RE-MESH: A SELF-HEALING MESH NETWORK ARCHITECTURE FOR RESILIENT POST-DISASTER URBAN COMMUNICATION

25-26J-011

Project Proposal Report

Dulaksha Tharindi Jayanath

B.Sc. (Hons) Degree in Information Technology

Specializing in Computer Systems and Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

RE-MESH: A SELF-HEALING MESH NETWORK ARCHITECTURE FOR RESILIENT POST-DISASTER URBAN COMMUNICATION

25-26J-011

Project Proposal Report

Dulaksha Tharindi Jayanath

IT22563446

Supervisor: Mr. Uditha Dharmakeerthi

Co- Supervisor: Mr. Amila Senarathne

B.Sc. (Hons) Degree in Information Technology

Specializing in Computer Systems and Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

DECLARATION OF THE CANDIDATE & SUPERVISOR

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	IT Number	Signature
Dulaksha Tharindi Jayanath	IT22563446	

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

Signature of the supervisor

.....
Mr. Uditha Dharmakeerthi

30/08/25
Date

Signature of the co-supervisor

.....
Mr. Amila Senarathne

30/08/25
Date

Abstract

Post-disaster communication systems break down when infrastructure fails. Sri Lanka gets hit by monsoon flooding every year, and when the waters rise, cellular towers go down, landline networks go offline, and the communities that need help most are the ones who lose contact first. The Re-Mesh project tackles this by building a self-healing mesh network that holds communication together in exactly those conditions. This proposal describes one component within that larger system: a flood prediction subsystem that runs entirely on the Re-Mesh nodes and gives communities an early warning without needing any external network.

The flood prediction component integrates six environmental sensors temperature, relative humidity, water flow rate, rainfall intensity, water turbidity, and pH level, directly onto the ESP32-based Re-Mesh nodes. Sensor readings feed continuously into a machine learning prediction engine that trains and compares XGBoost and Logistic Regression classifiers. The better-performing model is then refined through GridSearchCV hyperparameter tuning and deployed on the device to classify flood risk in real time.

Results and emergency notices appear on a lightweight web dashboard hosted directly on each ESP32 node. Anyone on the local Wi-Fi can pull it up using the node's IP address no internet required. This means the flood warning keeps working precisely when traditional communication has stopped. By embedding predictive intelligence into the mesh nodes themselves, this component shifts Re-Mesh from a reactive communication tool to a proactive early-warning platform for Sri Lankan urban flood scenarios.

Keywords: Flood Prediction, Machine Learning, XGBoost, Logistic Regression, Mesh Networks, IoT Sensors, Disaster Early Warning, ESP32, Self-Healing Networks, Smart Dashboard

TABLE OF CONTENTS

Declaration of the Candidate & Supervisor i

Abstract ii

Table of Contents iii

List of Figures v

List of Tables vi

List of Abbreviations vii

1. Introduction 1

1.1 Background & Literature Survey 1

1.1.1 Introduction to Disaster Communication Challenges 1

1.1.2 The Re-Mesh System and the Role of This Component 1

1.1.3 Current Technology Solutions and Limitations 2

1.2 Research Gap 3

1.3 Research Problem 4

2. Objectives 5

2.1 Main Objective 5

2.2 Specific Objectives 5

3. Methodology 8

3.1 System Architecture 8

3.2 Hardware Architecture and Technologies 10

3.3 Data Collection and Preprocessing Pipeline 12

3.4 Machine Learning Model Training and Selection 13

3.5 Hyperparameter Optimisation with GridSearchCV 13

3.6 Edge Deployment on ESP32 14

3.7 Built-in Web Dashboard 14

3.8 Research Approach 15
3.8.1 Time Frame 15
3.8.2 Systematic Development and Validation Approach 18
4. Project Requirements 20
4.1 Functional Requirements 20
4.2 Non-Functional Requirements 21
4.2.1 Performance Requirements 21
4.3 Expected Test Cases 22
5. Description of Personal and Facilities 26
5.1 Personal Qualifications and Expertise 26
5.2 Facility Requirements and Availability 27
6. Budget and Budget Justification 29
6.1 Estimated Budget for Flood Prediction Component 29
6.2 Budget Flexibility 31
6.3 Cost Optimization 31
7. Commercialization of the Product 32
7.1 Target Market 32
7.2 Market Size and Opportunity 32
7.3 Competitive Landscape 32
7.4 Business Model 33
8. Conclusions and Recommendations 34
8.1 Conclusion 34
8.2 Recommendations 34
8.3 Future Enhancements 35
9. Reference List 37

LIST OF FIGURES

Figure 3.1 : System Diagram 8
Figure 3.2 : Block Diagram 10
Figure 3.3: Gantt Chart 1 16
Figure 3.4 : Gantt Chart 2 16

LIST OF TABLES

Table 3.1 : Selected Environmental Sensors and Their Roles in Flood Prediction 11
Table 3.2 : Gantt Chart – Part I (Months 1–6) 16
Table 3.3 :Gantt Chart – Part II (Months 7–12) 17
Table 4.1 : Expected Test Cases for the Flood Prediction Component 22
Table 5.1 : Technical Skills and Their Relevance to the Flood Prediction Component 26
Table 6.1 : Estimated Budget and Justification for the Flood Prediction Component29

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
DHT22	Digital Humidity and Temperature Sensor (Model 22)
DMC	Disaster Management Centre
ESP32	Espressif Systems 32-bit Microcontroller
F1-score	Harmonic Mean of Precision and Recall
FR	Functional Requirement
GIS	Geographic Information System
GPIO	General Purpose Input/Output
IoT	Internet of Things
ISM	Industrial, Scientific, and Medical (frequency band)
LKR	Sri Lankan Rupee
LoRa	Long Range Radio
ML	Machine Learning
NTU	Nephelometric Turbidity Unit

OEM	Original Equipment Manufacturer
pH	Potential of Hydrogen
RAM	Random Access Memory
SLIIT	Sri Lanka Institute of Information Technology
SRAM	Static Random Access Memory
TC	Test Case
TDS	Total Dissolved Solids
Wi-Fi	Wireless Fidelity
XGBoost	Extreme Gradient Boosting

1. INTRODUCTION

1.1 Background & Literature Survey

1.1.1 Introduction to Disaster Communication Challenges

Sri Lanka is highly susceptible to monsoon-driven flooding, which recurs annually and causes widespread destruction to infrastructure, displacement of communities, and significant loss of life. The 2021 and 2022 monsoon seasons, for example, resulted in extensive damage across multiple provinces, overwhelming both communication and emergency-response systems simultaneously. When flooding occurs, conventional communication infrastructure cellular towers, landline networks, and internet backbone links frequently suffers partial or total failure. This creates a dangerous gap: emergency responders and affected populations lose the ability to coordinate rescue operations, receive timely warnings, or access critical safety information at precisely the moments when such information is most needed. [2], [10]

Early warning systems play a pivotal role in mitigating the impact of floods. An effective warning system must detect the conditions that precede or accompany flooding and communicate the resulting risk assessment to communities in time for evacuation or protective action. Traditional early warning approaches in Sri Lanka have relied heavily on centralised meteorological stations and government broadcasting channels. However, these systems suffer from latency, limited spatial granularity, and dependency on functioning communication infrastructure all of which are compromised during the very events they are designed to warn about. [10].

1.1.2 The Re-Mesh System and the Role of This Component

The Re-Mesh project is a collaborative research initiative that aims to develop a self-healing mesh network architecture capable of maintaining communication in post-disaster urban environments. The overall system uses ESP32 microcontroller-based nodes that communicate with one another using LoRa for long-range inter-node connectivity, while simultaneously providing Wi-Fi access points that allow local users including emergency service personnel and affected civilians to connect and exchange information via a built-in dashboard. A central monitoring dashboard enables network

administrators to track the health of each node, detect failures, and coordinate replacement or repositioning of nodes as needed. [1]

Within this broader system, the component described in this proposal is responsible for flood prediction using machine learning**.** This component transforms the Re-Mesh nodes from passive communication relays into active environmental sensing and prediction platforms. By collecting real-time data from multiple environmental sensors mounted on each node and feeding that data into a trained machine learning model, this component generates continuous flood risk assessments. These assessments are surfaced to users through the node's built-in web dashboard, providing localised, up-to-the-minute flood risk information and auto-generated emergency notices — all without requiring internet connectivity.

1.1.3 Current Technology Solutions and Limitations

Existing flood prediction and early warning systems across the literature and in practice fall into several broad categories, each carrying notable limitations in the context of decentralised, infrastructure-independent deployment.

Centralised meteorological and hydrological monitoring systems, such as those operated by national disaster management agencies, rely on dedicated sensor networks that transmit data to remote data centres for processing. While these systems can cover large geographic areas, they suffer from high latency in prediction delivery, limited granularity at the neighbourhood level, and complete dependence on functioning communication and power infrastructure precisely the resources that floods tend to destroy. [10]

Satellite-based remote sensing and GIS platforms provide valuable macro-level flood mapping and post-event analysis, but they are not well suited to providing real-time, localised early warnings to individual communities. The processing pipelines involved introduce delays that can render warnings ineffective for imminent flood events. [10]

IoT-based environmental monitoring systems that use standalone sensor nodes to collect data such as water levels, rainfall, and soil moisture have shown significant promise. However, most research implementations transmit collected data to cloud-based

platforms for analysis, reintroducing the dependency on reliable internet connectivity. When network infrastructure fails as it commonly does during floods these systems lose their ability to deliver warnings at the critical moment. [9], [11]

Machine learning models for flood prediction, including ensemble methods such as XGBoost and classification models such as Logistic Regression, have demonstrated strong predictive performance in academic literature when trained on appropriate environmental datasets. However, the deployment of such models in edge computing environments where inference runs directly on resource-constrained devices without cloud connectivity remains an underexplored area, particularly in the context of embedded IoT platforms designed for disaster scenarios. [3], [8], [12]

1.2 Research Gap

The existing literature and deployed systems reveal a clear gap at the intersection of three requirements: real-time environmental sensing, on-device machine learning prediction, and infrastructure-independent delivery of flood risk information to end users.

First, while IoT sensor deployments for environmental monitoring are well documented, systems that integrate multiple complementary sensors including water flow, turbidity, and pH alongside the more commonly used temperature, humidity, and rainfall into a single compact node are rare. The inclusion of turbidity and pH as predictive features adds meaningful signal for flood detection in river and urban waterway contexts, yet these sensors are frequently absent from lightweight field deployments.

Second, although machine learning models for flood prediction have been extensively evaluated in academic settings, the practical deployment of such models on microcontroller-class hardware such as the ESP32, with real-time inference on streaming sensor data, has not been adequately addressed. The challenges of model optimisation, feature engineering for edge deployment, and latency-constrained inference on limited hardware remain open research problems.

Third, and critically for this project, there is a significant gap in systems that combine on-device prediction with a locally accessible, user-friendly interface that requires no external network connectivity. The Re-Mesh built-in dashboard provides exactly this

capability, but no prior work has demonstrated the integration of a trained flood prediction model with such a dashboard on an embedded mesh network node.

1.3 Research Problem

The primary research problem addressed by this component is:

"How can a machine learning-based flood prediction system be designed, trained, and deployed on ESP32-based Re-Mesh nodes to provide real-time, accurate flood risk assessment and emergency notifications to local users through an embedded dashboard, without dependency on external internet connectivity?"

This problem encompasses the full pipeline from multi-sensor data acquisition and preprocessing, through model training and hyperparameter optimisation, to edge deployment and dashboard integration all within the constraints of the Re-Mesh hardware platform and the operational realities of Sri Lankan flood scenarios.

2. OBJECTIVES

2.1 Main Objective

To design, develop, and deploy a sensor-driven flood prediction system using machine learning on ESP32-based Re-Mesh nodes that provides real-time flood risk assessment and emergency notifications to local users through a built-in web dashboard, fully operational without external internet connectivity.

This main objective encompasses the end-to-end development of the flood prediction component from the selection and integration of environmental sensors, through the collection, preprocessing, and analysis of sensor data, to the training and optimization of machine learning models and their deployment on the Re-Mesh edge platform. The objective specifically targets flood scenarios in Sri Lankan urban environments, where infrastructure failure coincides with the greatest need for accurate, accessible early-warning information.

2.2 Specific Objectives

The specific objectives detail the individual technical and functional components that must be successfully developed to achieve the main objective

- Design and implement a multi-sensor data acquisition system integrated with ESP32-based Re-Mesh nodes

This objective focuses on the selection, configuration, and integration of six environmental sensors temperature, relative humidity, water flow rate, rainfall intensity, water turbidity, and pH onto the ESP32 hardware platform used by Re-Mesh nodes. The sensor suite must be capable of continuous, reliable data collection under the environmental conditions typical of Sri Lankan urban flood scenarios. Data acquisition routines must be designed to minimise power consumption while maintaining the sampling frequency required for timely flood detection, and the collected data must be stored and made available for downstream preprocessing and model inference.

- Collect, preprocess, and prepare environmental sensor data for machine learning model training

This objective addresses the data engineering pipeline that transforms raw sensor readings into a clean, structured dataset suitable for training predictive models. Activities include the collection of historical and real-time sensor data, handling of missing values and sensor noise, feature engineering to extract predictive signals from the raw inputs, and the construction of a labelled training dataset that maps sensor readings to known flood and non-flood events. The quality and representativeness of this dataset will directly determine the predictive performance of the downstream models.

- Train and evaluate flood prediction models using XGBoost and Logistic Regression, and select the most accurate model

This objective covers the core machine learning development work. Two classification models XGBoost, an ensemble gradient-boosting algorithm known for strong performance on tabular data, and Logistic Regression, a well-interpretable baseline classifier will be trained on the prepared dataset. Both models will be evaluated using standard classification metrics including accuracy, precision, recall, and F1-score, and their performance will be compared to determine which model is most suitable for deployment in the Re-Mesh edge environment. Model interpretability, inference speed, and compatibility with the ESP32 hardware will also be considered in the selection process.

- Optimise the selected model's hyperparameters using GridSearchCV to maximise prediction accuracy

Following initial model evaluation, the selected model will undergo systematic hyperparameter tuning to improve its predictive performance. GridSearchCV will be employed to conduct an exhaustive search over a defined parameter grid, with cross-validation used at each point to provide robust performance estimates and reduce the risk of overfitting. The outcome of this objective is a fully optimised model configuration that achieves the highest achievable accuracy on the validation dataset while retaining the computational efficiency necessary for real-time inference on the ESP32 platform.

- Deploy the optimised flood prediction model on ESP32-based Re-Mesh nodes for real-time edge inference

This objective addresses the practical challenge of deploying a trained machine learning model onto a resource-constrained microcontroller. The optimised model must be converted into a format compatible with the ESP32's processing capabilities, and an inference pipeline must be implemented that accepts live sensor data, runs the prediction, and produces a flood risk classification in real time. The deployment must be validated to ensure that inference latency is low enough to support timely warning generation and that the model's resource consumption does not compromise other critical functions of the Re-Mesh node.

- Develop a built-in web dashboard on the ESP32 node that displays real-time flood risk levels and auto-generated emergency notices

The final specific objective concerns the user-facing interface through which flood risk information reaches the public. A lightweight web dashboard will be developed and hosted directly on the ESP32 node, accessible to any device on the local network via the node's IP address without requiring internet connectivity. The dashboard will display the current flood risk level as determined by the on-device prediction model and will present contextually appropriate emergency notices and safety guidance that are generated dynamically based on the predicted risk. The interface must be intuitive, responsive, and accessible to users of varying technical ability, including emergency responders and displaced civilians.

3. METHODOLOGY

3.1 System Architecture

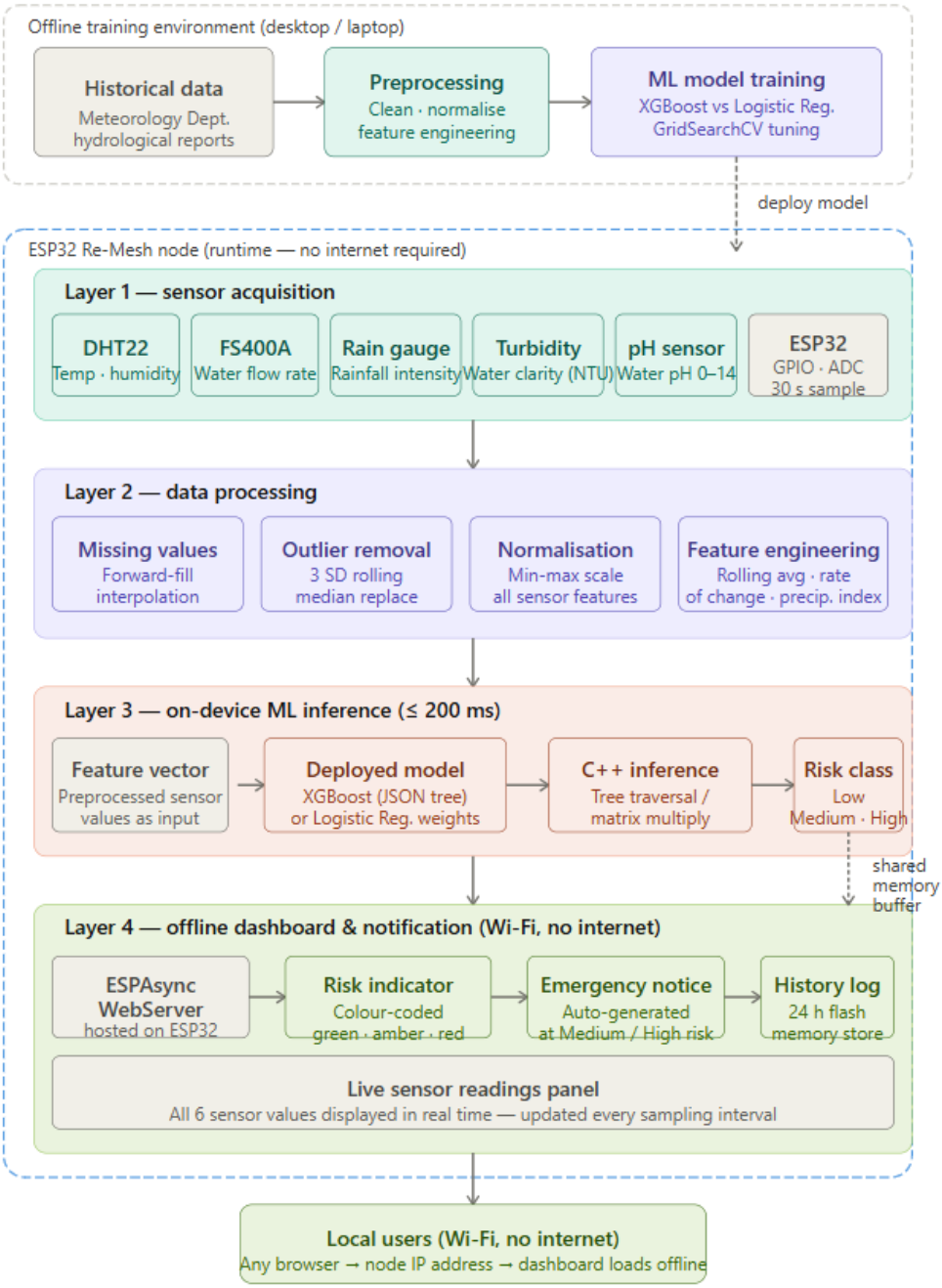

Figure 3.1 : System Diagram

The flood prediction component is organized into two distinct environments: an offline training environment that runs once on a development machine before deployment, and a four-layer runtime pipeline that runs continuously on each ESP32 Re-Mesh node without any internet connectivity.

Offline training environment. Before deployment, ten years of hydrological annual reports from the Sri Lanka Meteorology Department must be preprocessed cleaned, normalised, and feature-engineered and used to train two competing classifiers: XGBoost and Logistic Regression. The better-performing model must be refined through GridSearchCV hyperparameter tuning and then serialised into a compact, ESP32-compatible format. This trained model must deployed to the device once and thereafter runs entirely on-device.

Layer 1 — Sensor acquisition. Five sensor modules the DHT22 (temperature and humidity), FS400A (water flow rate), tipping-bucket rain gauge (rainfall intensity), turbidity sensor, and analog pH sensor connect to the ESP32 through its GPIO and ADC interfaces. All six environmental parameters are sampled every 30 seconds, timestamped by the ESP32's real-time clock, and buffered locally on the device.

Layer 2 — Data processing. Each raw sensor batch passes through four sequential operations: forward-fill interpolation handles short missing-value gaps; a 3-standard-deviation rolling check detects and replaces outliers with the rolling median; min-max normalisation rescales all features to a common range; and feature engineering derives rolling averages, rates of change, and a composite precipitation index. The resulting feature vector is passed upward to the inference layer.

Layer 3 — On-device ML inference. The deployed model — either an XGBoost JSON tree ensemble traversed by a C++ inference engine, or a Logistic Regression weight array evaluated by a single matrix multiplication — processes the feature vector and produces a flood-risk classification (Low, Medium, or High). The result is written to a shared memory buffer within the 200-millisecond latency target and made immediately available to the dashboard layer.

Layer 4 — Offline dashboard and notification. An ESPAsyncWebServer instance hosted on the ESP32 serves a web interface accessible to any device connected to the

node's Wi-Fi access point, at the node's local IP address, with no internet connection required. The dashboard renders a colour-coded risk indicator (green for Low, amber for Medium, red for High), auto-generated emergency notices at Medium and High risk levels, a 24-hour prediction history log stored in the ESP32's flash memory, and a live panel showing the most recent reading from all six sensors.

3.2 Hardware Architecture and Technologies

Figure 3.2 : Block Diagram

The hardware foundation of the flood prediction component is the same ESP32 microcontroller board used across all Re-Mesh nodes. The ESP32 was selected for its dual-core processor, substantial on-board RAM (520 KB SRAM), built-in Wi-Fi for dashboard serving, and extensive GPIO interface support all of which are essential for simultaneously running sensor acquisition, model inference, and a responsive web server. [17]

Six environmental sensors have been selected to provide a comprehensive and redundant set of input features for the flood prediction model. Each sensor was chosen based on its relevance to flood dynamics in Sri Lankan urban contexts, its physical compatibility with the ESP32 GPIO and ADC interfaces, its power consumption profile, and

its availability and cost in the local market. The table below summarises the selected sensors and their roles.

Table 3.1 : Selected Environmental Sensors and Their Roles in Flood Prediction

Sensor Module	Parameter	Accuracy / Range	Role in Flood Prediction
DHT22	Temperature & Humidity	±0.5 °C / ±2 % RH	Ambient air temperature and relative humidity – two of the most significant meteorological drivers of flood formation and progression.
FS400A	Water Flow Rate	0–30 L/min	Measures volumetric flow through waterways and drainage channels; a sudden rise in flow is a direct precursor indicator of downstream flooding.
Rain Gauge (Tipping-Bucket)	Rainfall Intensity	0.2 mm resolution	Quantifies precipitation rate in real time; sustained high-intensity rainfall is the primary trigger for flash and urban flooding.
TDS / Turbidity Sensor	Water Turbidity	0–100 NTU	Elevated turbidity correlates with increased sediment load carried by rising floodwaters, providing an independent confirmation signal.
Analog pH Sensor	Water pH	0–14	Rapid pH shifts in surface water can indicate unusual hydrological activity or industrial discharge preceding or accompanying a flood event.

The six sensors collectively capture the full causal chain of an urban flood event: precipitation triggers rising water flow and turbidity, while ambient temperature and humidity influence evaporation rates and ground saturation. pH monitoring provides an

additional independent signal that can help distinguish genuine flood conditions from other causes of flow increases. Together, these features give the machine learning model a rich and multi-dimensional input space from which to learn flood patterns.

3.3 Data Collection and Preprocessing Pipeline

Sensor data is sampled at a configurable interval initially set at 30 seconds and each reading is timestamped by the ESP32's real-time clock. Raw readings are buffered locally on the device and, where historical connectivity is available, also transmitted to a central data store for the purpose of building the training dataset. The preprocessing pipeline performs the following operations on the collected data.

Missing value handling:

Intermittent sensor failures or communication gaps can produce null readings. Missing values are identified and handled using forward-fill interpolation for short gaps (up to three consecutive missing samples) and flagged for exclusion where gaps exceed the interpolation threshold.

Outlier detection and removal:

Statistical outliers values that deviate beyond three standard deviations from the rolling mean of a given sensor are identified and replaced with the rolling median. This step protects the model from being distorted by sensor noise or transient electrical interference.

Normalisation:

All sensor features are scaled to a common range using min-max normalisation. This ensures that features with inherently larger numerical ranges (such as flow rate) do not dominate the model's learned weights over features with smaller ranges (such as pH).

Feature engineering:

In addition to the raw (preprocessed) sensor values, several derived features are computed to enrich the input space. These include rolling averages (over 5- and 15-minute windows) for each sensor, the rate of change of water flow and turbidity between consecutive samples, and a composite precipitation index combining rainfall intensity

with humidity. These derived features capture temporal trends and interactions that the raw instantaneous readings alone cannot convey. [11], [14]

3.4 Machine Learning Model Training and Selection

Two classification models are trained and compared to identify the most suitable algorithm for real-time flood prediction on the ESP32 edge platform **XGBoost** and **Logistic Regression**.

XGBoost is an ensemble gradient-boosting framework that constructs an additive sequence of decision trees, each one correcting the errors of its predecessor. It is well suited to tabular sensor data, is robust to feature scale differences, can capture non-linear relationships between features, and provides built-in feature importance scores that aid interpretability. Its primary trade-off for edge deployment is inference speed: larger trees and deeper ensembles increase the per-prediction computation time. [9]

Logistic Regression is a linear binary classifier that models the probability of the flood class as a sigmoid function of a weighted sum of the input features. It is computationally lightweight making it an attractive candidate for microcontroller deployment and its coefficient weights provide direct interpretability of each feature's contribution to the prediction. However, it assumes a linear decision boundary and may underperform XGBoost where feature interactions are significant. [10]

Both models are trained on the same preprocessed and feature-engineered dataset using an 80/20 train-test split. Performance is evaluated using accuracy, precision, recall, F1-score, and the area under the ROC curve (AUC-ROC). The model that achieves the highest F1-score on the test set the metric most appropriate for imbalanced flood/non-flood data is selected for hyperparameter optimisation and subsequent deployment. [11], [19]

3.5 Hyperparameter Optimisation with GridSearchCV

Once the best-performing model is identified, its predictive performance is further refined through systematic hyperparameter tuning. GridSearchCV is employed to perform an exhaustive grid search across a predefined range of candidate hyperparameter values.

For **XGBoost**, the search grid spans key parameters including the number of estimators (trees), maximum tree depth, learning rate, minimum child weight, and subsample ratio. For **Logistic Regression**, the grid covers the regularisation strength (inverse of C), the regularisation type (L1 versus L2), and the solver algorithm.

At each point in the grid, five-fold cross-validation is performed on the training set. The fold with the highest mean F1-score across the five folds is retained, and the hyperparameter configuration that produces the best mean cross-validated F1-score is selected as the final model configuration. This procedure minimises the risk of overfitting while maximising generalisation to unseen flood events. [12]

3.6 Edge Deployment on ESP32

The optimised model is serialised and converted into a format that can be loaded and executed on the ESP32 at inference time. For XGBoost, this involves exporting the trained model to a compact JSON or binary representation and implementing a lightweight inference engine in C++ that traverses the decision-tree ensemble on the microcontroller. For Logistic Regression, deployment is simpler: the model reduces to a fixed set of weights and a bias term, which are stored as a small array and applied via a single matrix multiplication at each prediction cycle.

The inference pipeline on the ESP32 operates as follows: the sensor acquisition routine delivers a new set of preprocessed and feature-engineered values at each sampling interval; the inference engine loads these values, runs the model forward pass, and outputs a predicted flood-risk class (Low, Medium, or High); and the result is written to a shared memory buffer that the dashboard web server reads to update the displayed risk level. End-to-end inference latency is targeted to remain below 200 milliseconds to ensure that the dashboard reflects the latest prediction within a single sampling cycle. [13], [18]

3.7 Built-in Web Dashboard

A lightweight web application is hosted directly on the ESP32 using its built-in Wi-Fi access point and a minimal HTTP server (e.g., ESPAsyncWebServer). Users connect to the node's Wi-Fi network and navigate to the node's IP address in any standard

browser no internet connection is required. The dashboard presents the following information in real time.

Current Flood Risk Level: Displayed as a colour-coded indicator (green for Low, amber for Medium, red for High) that updates automatically each time a new prediction is produced by the on-device model.

Live Sensor Readings: A panel showing the most recent value from each of the six sensors, updated at the sampling interval, so that users can see the raw environmental conditions alongside the model's interpretation of those conditions.

Emergency Notices: When the predicted risk level reaches Medium or High, the dashboard automatically generates and displays a contextual emergency notice. The notice content is determined by the risk level and includes guidance appropriate to the severity from advisory warnings at Medium risk to evacuation instructions at High risk. Notice text is stored on the device and selected programmatically; no external content service is involved. [20]

Prediction History: A simple time-series log of recent risk-level predictions is maintained in the ESP32's flash memory and displayed on the dashboard, allowing users to observe how the flood risk has evolved over the preceding hours.

3.8 Research Approach

3.8.1 Time Frame

The project is organised into a twelve-month development timeline divided into five sequential phases. Each phase builds directly on the outputs of the phase that precedes it, and key validation checkpoints are embedded at phase boundaries to ensure that any issues are identified and resolved before work progresses. The two Gantt charts below illustrate the planned schedule across the full twelve-month period.

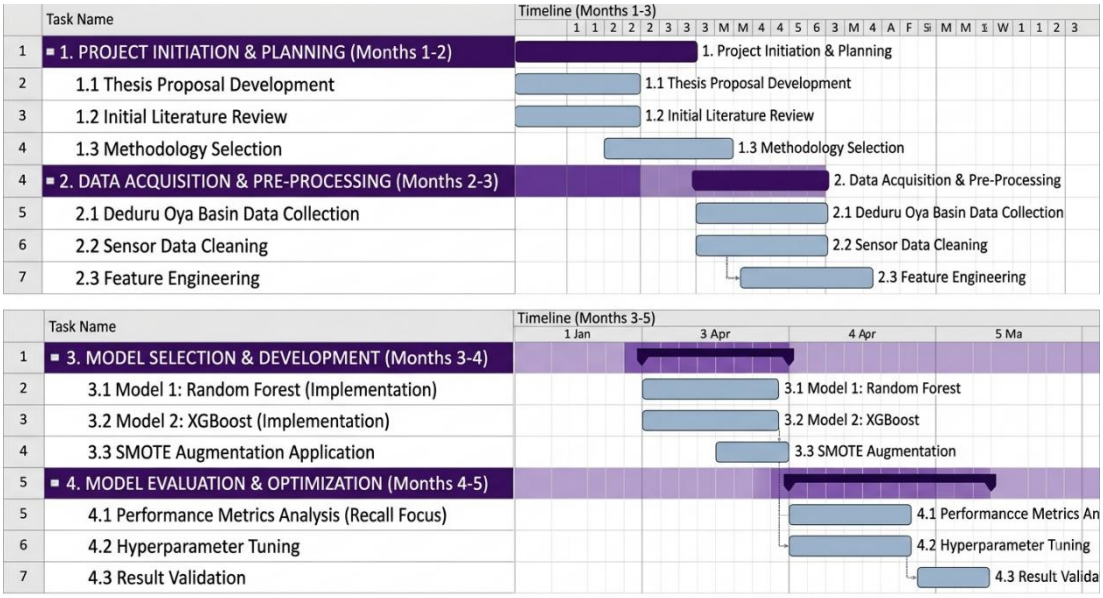

Figure 3.3: Gantt Chart 1

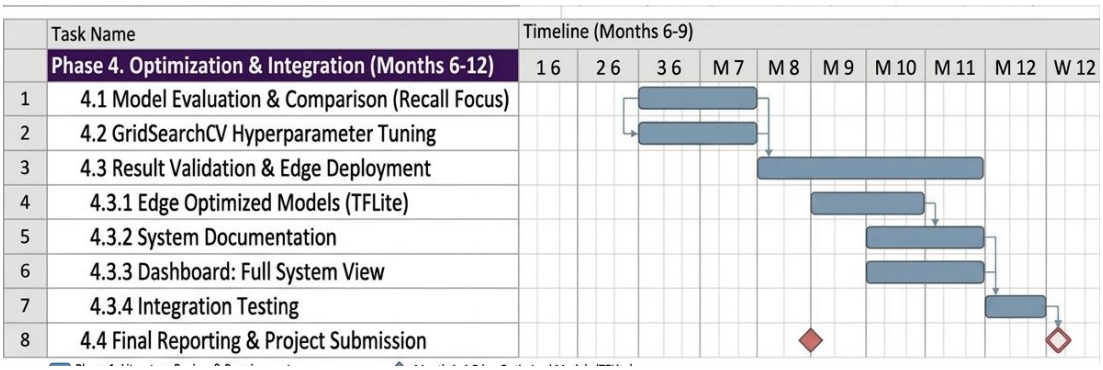

Figure 3.4 : Gantt Chart 2

Table 3.2 : Gantt Chart – Part I (Months 1–6)

Phase / Activity	M1	M2	M3	M4	M5	M6
Literature Review & Requirements Analysis	●	●				
Sensor Selection & Hardware Procurement	●	●				

ESP32 Sensor Integration & Data Acquisition		●	●			
Historical Data Collection & Preprocessing		●	●	●		
Feature Engineering & Dataset Preparation			●	●		
ML Model Training (XGBoost & Logistic Reg.)				●	●	
Model Evaluation & Comparison					●	

Table 3.3 :Gantt Chart – Part II (Months 7–12)

Phase / Activity	M7	M8	M9	M10	M11	M12
GridSearchCV Hyperparameter Optimisation	●					
Optimised Model Edge Deployment on ESP32	●	●				
Dashboard Development & Integration		●	●			
Unit & Integration Testing			●	●		
Field / Simulation Testing				●	●	
Performance Benchmarking & Validation					●	●
Documentation & Report Writing					●	●

3.8.2 Systematic Development and Validation Approach

- Phase 1 — Literature Review and Requirements Analysis (Months 1–2)

A thorough review of existing flood prediction systems, IoT sensor platforms, machine learning approaches for environmental forecasting, and edge deployment techniques is conducted. Concurrently, the specific functional and non-functional requirements for the flood prediction component are defined in consultation with the wider Re-Mesh team, with particular attention to the constraints imposed by the ESP32 hardware and the offline-dashboard deployment model.

- Phase 2 — Sensor Integration and Data Collection (Months 2–4)

The six selected sensors are physically integrated onto ESP32 Re-Mesh nodes, firmware is developed for reliable continuous sampling, and the data acquisition and local storage pipeline is implemented and tested. Historical environmental data — sourced from meteorological records, open datasets, and initial field readings — is collected and begins to accumulate for subsequent preprocessing and model training.

- Phase 3 — Data Preprocessing and Model Training (Months 4–7)

The collected dataset is processed through the full preprocessing pipeline described in Section 3.3. Feature engineering is performed, and the labelled training dataset is assembled. Both XGBoost and Logistic Regression models are trained, evaluated, and compared. The winning model is then subjected to GridSearchCV hyperparameter optimisation as described in Section 3.5.

- Phase 4 — Edge Deployment and Dashboard Development (Months 7–9)

The optimised model is converted to its edge-deployable format and integrated into the ESP32 inference pipeline. The web dashboard application is developed, integrated with the prediction and sensor-reading outputs, and tested for responsiveness and correctness of display. End-to-end latency from sensor reading to dashboard update is measured and optimised.

- Phase 5 — Testing, Validation, and Documentation (Months 9–12)

Comprehensive unit and integration testing is performed against the test cases defined in Section 4.3. Simulated flood scenarios are run to validate the model's predictive accuracy under realistic conditions, and the dashboard's emergency-notice generation is verified across all risk levels. Performance benchmarking confirms that the system meets the targets stated in Section 4.2. Full technical documentation and the final project report are completed.

4. PROJECT REQUIREMENTS

4.1 Functional Requirements

FR-01: Continuous Multi-Sensor Data Acquisition

The system shall continuously acquire readings from all six environmental sensors (temperature, humidity, water flow, rainfall, turbidity, and pH) at a configurable sampling interval. Each reading shall be timestamped and buffered locally on the ESP32 node. No reading shall be discarded without explicit preprocessing justification (e.g., confirmed outlier).

FR-02: Automated Data Preprocessing

The system shall automatically apply the preprocessing pipeline including missing-value interpolation, outlier detection and replacement, min-max normalisation, and feature engineering to every batch of incoming sensor data before it is passed to the inference engine. The preprocessing parameters (e.g., outlier threshold, normalisation bounds) shall be configurable without requiring a firmware rebuild.

FR-03: Real-Time Flood Risk Classification

The system shall execute the optimised flood prediction model on each new set of preprocessed features and produce a flood-risk classification (Low, Medium, or High) within 200 milliseconds of the feature vector becoming available. The classification result shall be immediately made accessible to the dashboard layer.

FR-04: Dynamic Emergency Notice Generation

When the predicted flood-risk level is Medium or High, the system shall automatically generate and display an appropriate emergency notice on the dashboard. The notice content shall be determined by the current risk level according to a predefined mapping stored on the device. Notices shall update in real time as the risk level changes; no manual intervention shall be required.

FR-05: Built-in Web Dashboard

The system shall host a web dashboard on the ESP32 node, accessible via the node's local IP address over Wi-Fi without any internet connectivity. The dashboard shall display, at a minimum: the current flood-risk level with colour-coded visual indicator, live readings from all six sensors, the most recent emergency notice (if any), and a time-series log of recent risk-level predictions.

FR-06: Prediction History Logging

The system shall maintain a local log of flood-risk predictions in the ESP32's flash memory. The log shall retain at least the most recent 24 hours of predictions with their timestamps and shall be readable through the dashboard interface.

4.2 Non-Functional Requirements

4.2.1 Performance Requirements

The end-to-end latency from a new sensor reading being acquired to the corresponding risk prediction appearing on the dashboard shall not exceed one full sampling interval (target: 30 seconds). The model inference step alone shall complete within 200 milliseconds. The dashboard web server shall be capable of serving at least five simultaneous client connections without noticeable degradation in responsiveness.

4.2.2 Accuracy and Reliability

The deployed flood prediction model shall achieve an F1-score of no less than 0.85 on the held-out test dataset. The system shall not crash or enter an unrecoverable error state due to a single sensor failure; in such an event, the affected sensor's contribution shall be gracefully degraded (via interpolation or exclusion) while the remaining sensors continue to supply the model. The dashboard shall accurately reflect the model's output at all times; stale predictions shall not persist beyond one sampling cycle.

4.2.3 Power and Resource Efficiency

The flood prediction component shall not consume more than 30% of the ESP32's available processing capacity averaged over any five-minute window, so as to leave sufficient headroom for the Re-Mesh node's other functions (LoRa communication, Wi-Fi access point, central-dashboard reporting). The sensor acquisition and

preprocessing routines shall be designed to minimise unnecessary wake cycles and memory allocations on the microcontroller.

4.2.4 Usability and Deployment

The dashboard interface shall be operable by a user with no technical background. Risk levels shall be communicated primarily through colour coding and plain-language labels. Emergency notices shall be written in clear, actionable language. The entire system shall be deployable by a trained technician in under 30 minutes, including sensor mounting, ESP32 configuration, and dashboard verification.

4.2.5 Maintainability and Configurability

Key system parameters including sensor sampling interval, outlier-detection thresholds, normalisation bounds, risk-level thresholds, and emergency-notice text — shall be stored in a configuration file on the ESP32's flash storage and shall be updatable without recompiling the firmware. The codebase shall be documented sufficiently for another developer to understand, modify, and extend any component of the pipeline.

4.3 Expected Test Cases

Ten test cases have been defined to cover the full scope of the flood prediction component, from individual unit-level checks through to end-to-end simulated flood scenarios. Each test case specifies a clear procedure and an unambiguous expected outcome against which the system's behaviour can be objectively verified. The table below summarises all planned test cases.

Table 4.1 : Expected Test Cases for the Flood Prediction Component

ID	Test Description	Procedure	Expected Outcome
TC-01	Sensor Data Acquisition Continuity	Power on all six sensors; run acquisition loop for 24 hours continuously; verify no null readings are	All readings recorded; zero unrecoverable nulls

		produced and all timestamps are sequential.	over the 24-hour window.
TC-02	Preprocessing Pipeline – Outlier Handling	Inject known outlier values (>3 SD) into the raw stream; run the preprocessing pipeline; inspect the output dataset.	Outliers are replaced with rolling-median values; no outlier propagates into the feature-engineered dataset.
TC-03	Model Prediction – Accuracy on Test Set	Load the optimised model; run inference on the held-out test set; compute accuracy, precision, recall, and F1-score.	F1-score meets or exceeds the target threshold established during model selection; no class is completely misclassified.
TC-04	GridSearchCV – Best Hyperparameter Recovery	Execute GridSearchCV with a known small grid; compare the selected configuration against manually verified best parameters.	The reported best configuration matches the manually verified optimum; cross-validation scores are correctly averaged.
TC-05	Edge Inference Latency	Deploy the optimised model on ESP32; feed 100 consecutive sensor-	Mean per-prediction latency remains below 200

		reading vectors; measure wall-clock time per inference.	ms; no single inference exceeds 500 ms.
TC-06	Dashboard Risk-Level Display Update	Simulate sequential model outputs of Low → Medium → High risk; observe the dashboard in a connected browser.	Colour-coded indicator and risk label update in real time for each transition; no stale values persist beyond one sampling cycle.
TC-07	Emergency Notice Generation	Trigger model outputs at Medium and High risk levels; observe the dashboard notification panel.	Correct advisory or evacuation notice is displayed for each risk level; notice content matches the predefined mapping.
TC-08	Simulated Flood Scenario – End-to-End	Feed a historical flood-event sensor trace through the full pipeline (acquisition → preprocessing → inference → dashboard) in real time.	The dashboard correctly escalates from Low to High risk during the flood portion of the trace and returns to Low afterwards; all

			intermediate transitions are logged.
TC-09	Node Power-Loss Resilience	Simulate a power interruption to one sensor mid-acquisition; verify the system's behaviour.	Missing values from the interrupted sensor are handled via forward-fill; remaining sensors continue uninterrupted; no system crash occurs.
TC-10	Dashboard Accessibility Without Internet	Connect a client device to the ESP32 Wi-Fi access point with no upstream internet; attempt to load the dashboard.	Dashboard loads fully; all panels (risk indicator, sensor readings, notices, history) render correctly without any external resource dependency.

5. DESCRIPTION OF PERSONAL AND FACILITIES

5.1 Personal Qualifications and Expertise

The principal investigator is a final-year B.Sc. (Hons) student in Information Technology, specialising in Computer Systems and Network Engineering at the Sri Lanka Institute of Information Technology (SLIIT). The following table summarises the technical skills directly relevant to the flood prediction component of the Re-Mesh project.

Table 5.1 : Technical Skills and Their Relevance to the Flood Prediction Component

Skill Category	Relevance to Flood Prediction Component	Proficiency Level
Python Programming	Core language for data preprocessing, feature engineering, ML model training (XGBoost, Logistic Regression), and GridSearchCV optimisation	Advanced
Machine Learning & Data Science	Essential for model design, training, evaluation, hyperparameter tuning, and comparative classifier analysis using scikit-learn and XGBoost libraries	Advanced
Embedded Systems (ESP32)	Required for sensor integration, firmware development, edge model deployment, and lightweight web-server hosting on the microcontroller	Intermediate
IoT Sensor Integration	Fundamental for selecting, wiring, calibrating, and reading the six environmental sensors via GPIO and ADC interfaces	Intermediate

Web Development (HTML/CSS/JS)	Necessary for designing and implementing the built-in dashboard UI, including colour-coded risk indicators and dynamic emergency notices	Intermediate
Data Preprocessing & Feature Engineering	Critical for cleaning raw sensor streams, handling missing values and outliers, normalisation, and constructing derived temporal features	Advanced
Version Control (Git)	Used for collaborative development, code review, and maintaining a reproducible project history throughout all phases	Intermediate

5.2 Facility Requirements and Availability

The following facilities are required for the development, training, and testing of the flood prediction component. All items listed are either already available or readily accessible through the institution and the wider Re-Mesh project team.

- Development Workstations: Personal laptops equipped with sufficient processing power and RAM for Python-based machine learning workflows, including model training and GridSearchCV optimisation runs

- Development IDEs: Visual Studio Code or equivalent IDEs with Python, PlatformIO, and Arduino IDE support for embedded firmware development on the ESP32 platform

- Machine Learning Libraries: Python libraries including scikit-learn, XGBoost, pandas, NumPy, and matplotlib, all installable via standard package managers without institutional barriers

- Simulation and Testing Tools: Network simulation software for validating the dashboard's behaviour under varying connectivity conditions and for replaying historical flood-event sensor traces

- Hydrological Data Access: Access to the Sri Lanka Meteorology Department's hydrological annual reports covering the past ten years, used as the primary source for assembling the labelled training dataset
- Communication Tools: Microsoft Teams, WhatsApp, or similar platforms for coordination with the supervisor, co-supervisor, and the wider Re-Mesh project team
- ESP32 Development Boards and Sensors: Hardware components as itemised in Section 6 (Budget), procured through the project budget allocation

6. BUDGET AND BUDGET JUSTIFICATION

6.1 Estimated Budget for Flood Prediction Component

The table below presents the estimated costs for all hardware and tooling required by the flood prediction component. Costs are quoted in Sri Lankan Rupees (LKR) and reflect current market prices for components available locally. Where price ranges are given, the lower bound assumes bulk purchasing through the Re-Mesh project and the upper bound reflects individual retail acquisition.

Table 6.1 : Estimated Budget and Justification for the Flood Prediction Component

Component	**Estimated Cost (LKR)**	**Quantity**	**Justification**
ESP32 Development Boards	3 000 – 5 000	2	Primary microcontroller platform for sensor integration, edge model inference, and dashboard hosting. Two boards are required: one for active development and one for parallel testing.
DHT22 Temperature & Humidity Sensor	400 – 700	2	Provides ambient temperature and relative humidity readings, two key meteorological inputs to the flood prediction model. A spare unit is included for redundancy testing.
FS400A Water Flow Sensor	800 – 1 200	2	Measures volumetric water flow through drainage channels. A rapid rise in flow rate is a direct precursor to downstream flooding and

			constitutes one of the strongest predictive features.
Tipping-Bucket Rain Gauge	600 – 1 000	1	Quantifies real-time rainfall intensity. Sustained high-intensity precipitation is the primary trigger for flash and urban flooding in Sri Lankan monsoon scenarios.
TDS / Turbidity Sensor	350 – 600	2	Elevated water turbidity correlates with increased sediment transport during flood events. Two units allow comparative calibration and redundancy.
Analog pH Sensor	300 – 500	1	Detects unusual pH shifts in surface water that may accompany or precede flood-related hydrological activity.
Prototyping Materials (breadboards, wires, enclosures)	500 – 900	1 (set)	Required for physically assembling and housing the sensor–ESP32 prototype, including weatherproof enclosure material for simulated outdoor testing.
Power Supply & Battery Pack	600 – 1 000	1	Provides stable power to the ESP32 and sensor array during extended testing cycles, enabling power-consumption benchmarking as specified in Section 4.2.3.

The total estimated budget for the flood prediction component ranges from approximately LKR 6 550 to LKR 11 900. Software tools like Python, scikit-learn, XGBoost, PlatformIO, and Visual Studio Code are all open-source and incur no licence cost.

6.2 Budget Flexibility

- Actual component costs may vary based on supplier availability and current market conditions; prices quoted reflect estimates as of mid-2025
- Bulk purchasing of shared components (particularly ESP32 boards) through coordination with the wider Re-Mesh project team will reduce per-unit costs
- International shipping costs will be incurred only if locally unavailable components must be sourced; this contingency is already reflected in the upper bounds of the ranges above
- Final procurement decisions will be made in consultation with the supervisor to ensure optimal allocation of shared project resources

6.3 Cost Optimization

- Bulk Purchasing: Sensor modules and ESP32 boards that are common across multiple Re-Mesh components will be ordered together to obtain volume discounts
- Resource Sharing: Shared development workstations and testing equipment within the project team eliminate the need for duplicate hardware acquisitions
- Open-Source Software: The entire ML and firmware toolchain is built on open-source libraries, removing any software licensing expenditure
- Iterative Prototyping: Early-phase testing uses breadboard assemblies before final enclosure fabrication, reducing material waste from design iterations

7. COMMERCIALIZATION OF THE PRODUCT

7.1 Target Market

• National Disaster Management Agencies: Organisations such as the Sri Lanka Disaster Management Centre (DMC) that require localised, infrastructure-independent early-warning capabilities during flood events

• Municipal and Provincial Governments: Local authorities responsible for urban flood preparedness and community safety in flood-prone regions across Sri Lanka and neighbouring South Asian countries

• Agricultural and Water-Resource Authorities: Agencies that monitor river basins, irrigation channels, and watershed health, for which real-time turbidity, flow, and pH data add operational value beyond flood warning

• Insurance and Risk-Assessment Companies: Firms that assess and price flood risk for urban properties and infrastructure, where granular, real-time environmental data can improve actuarial models

7.2 Market Size and Opportunity

The global market for IoT-based environmental monitoring and early-warning systems is growing rapidly, driven by increasing frequency and severity of extreme weather events and by expanding government mandates for disaster preparedness. Within South Asia alone, annual flood losses exceed several billion US dollars, and the demand for affordable, deployable early-warning infrastructure is substantial. The flood prediction component of Re-Mesh addresses a specific but high-value niche: edge-deployable, infrastructure-independent flood prediction at the neighbourhood scale — a capability currently absent from commercially available solutions.

7.3 Competitive Landscape

Direct Competitors:

Cloud-connected IoT flood-monitoring platforms (e.g., Flood-IQ, FloodSmart) and national early-warning dashboards that rely on centralised data processing and internet connectivity.

Indirect Competitors:

Satellite-based flood mapping services (e.g., Copernicus Emergency Management Service), traditional meteorological broadcasting, and SMS-based alert systems operated by government agencies.

Competitive Advantages:

The Re-Mesh flood prediction component offers three principal advantages over existing alternatives. First, it operates entirely without internet connectivity, making it functional precisely when infrastructure fails. Second, it provides neighbourhood-level granularity by embedding prediction directly on each mesh node, whereas most competing systems offer only regional or city-level forecasts. Third, the use of low-cost, commercially available ESP32 hardware and open-source ML libraries results in a unit cost significantly below that of proprietary environmental monitoring stations.

7.4 Business Model

Hardware Sales: Pre-assembled flood-prediction mesh nodes, including the ESP32 board, all six sensors, enclosure, and power system, sold as plug-and-play units to end-user organizations.

Software and Model Licensing: The trained flood prediction models and the preprocessing and inference firmware stack can be licensed to OEMs or system integrators who wish to embed the capability in their own IoT platforms.

Subscription-Based Model Updates: Periodic retraining of the prediction model using updated hydrological data from new monsoon seasons can be offered as an annual subscription, ensuring that deployed units remain accurate as climate patterns evolve.

Consulting and Integration Services: Technical support for site selection, sensor calibration, network topology planning, and dashboard customization, targeted at municipal and humanitarian organizations deploying Re-Mesh in the field.

Training and Certification: Structured training programs for maintenance technicians and emergency-response personnel, covering sensor servicing, firmware updates, and dashboard interpretation.

8. CONCLUSIONS AND RECOMMENDATIONS

8.1 Conclusion

This proposal presents the flood prediction component of the Re-Mesh self-healing mesh network architecture, targeting post-disaster urban communication and early warning in Sri Lankan flood scenarios. The component addresses a critical gap in the existing landscape of flood early-warning systems: the absence of a fully infrastructure-independent, edge-deployable prediction capability that can deliver real-time, neighbourhood-level flood risk assessments without reliance on cloud connectivity or functioning telecommunications infrastructure.

The proposed system integrates six complementary environmental sensors onto ESP32-based Re-Mesh nodes and feeds their continuous readings into a machine learning prediction engine. The training dataset is assembled from hydrological annual reports published by the Sri Lanka Meteorology Department over the past ten years, providing a historically grounded and climatically representative labelled dataset for model development. Two classification algorithms — XGBoost and Logistic Regression — are trained, evaluated, and compared, and the superior model is further refined through systematic GridSearchCV hyperparameter optimisation before deployment on the edge platform. [8]

The optimised model runs in real time on the ESP32, producing flood-risk classifications (Low, Medium, or High) within 200 milliseconds of each new sensor reading. Results are surfaced through a lightweight web dashboard hosted on the node itself, accessible to any local device via Wi-Fi without internet connectivity. The dashboard displays colour-coded risk indicators, live sensor readings, dynamically generated emergency notices, and a prediction history log, making critical safety information available to emergency responders and affected civilians at the precise moments when external communication infrastructure has failed.

8.2 Recommendations

- Adopt a phased development approach, beginning with sensor integration and data collection before progressing to model training and edge deployment, to

ensure that each pipeline stage is thoroughly validated before downstream work begins

- Conduct all initial model training and hyperparameter optimisation on desktop hardware to enable rapid iteration, and transition to ESP32-based inference benchmarking only once the model configuration has stabilised
- Supplement the ten-year hydrological annual dataset with real-time sensor readings collected during the prototype deployment phase to enable online model adaptation and improve prediction accuracy over time
- Engage local emergency-response personnel in early usability testing of the dashboard to ensure that the interface meets the information-access needs of its intended users under high-stress conditions
- Coordinate closely with the wider Re-Mesh project team throughout all phases to ensure that the flood prediction component integrates seamlessly with the mesh network's communication, power management, and central monitoring subsystems

8.3 Future Enhancements

Deep Learning and Temporal Models: Once a sufficiently long real-time sensor dataset has been accumulated, LSTM or Transformer-based sequence models could be explored to capture longer-range temporal dependencies in the environmental data, potentially improving prediction lead time beyond what the current feature-window approach achieves

Federated Learning Across Mesh Nodes: As multiple Re-Mesh nodes are deployed in the field, a federated learning framework could allow each node to contribute locally collected data to a shared model improvement process without transmitting raw sensor readings to a centralised server, preserving data locality while continuously improving prediction accuracy across the network

Multi-Hazard Extension: The sensor-driven prediction architecture described in this proposal could be extended to detect and classify hazards beyond flooding such as

landslides or water-quality contamination events by retraining the model on appropriately labelled datasets and adding targeted sensors to the node.

Satellite Connectivity Integration: In scenarios where satellite communication links become available on Re-Mesh nodes (as explored in the wider project), the flood prediction component could transmit aggregated risk data to regional coordination centres, enabling city-scale flood-risk dashboards that combine neighbourhood-level predictions from multiple nodes.

9. REFERENCE LIST

[1] M. Grahovac, "Emergency network deployment: Mesh networks lifesaving power in disaster management," Meshmerize, 19 Oct. 2023. [Online]. Available: https://meshmerize.net/emergency-network-deployment-mesh-in-disaster-management/. [Accessed: 22 Aug. 2025].

[2] "Meshtastic," Meshtastic.org. [Online]. Available: https://meshtastic.org/docs/introduction/. [Accessed: 22 Aug. 2025].

[3] R. Hakani, A. Mishra, N. Shah, and A. Rawat, "Enhancing situational awareness with LoRa mesh networks: Communication in internet-deprived areas," Research Square, 2024.

[4] B. Karaman et al., "Solutions for sustainable and resilient communication infrastructure in disaster relief and management scenarios," arXiv [cs.NI], 2024.

[5] M. A. Ertürk, M. A. Aydin, L. Vollero, and R. Setola, "IEEE 802.11s mesh network analysis for post disaster communication," in Lecture Notes in Electrical Engineering, Singapore: Springer Singapore, 2019, pp. 53–59.

[6] "Why Meshtastic uses managed flood routing," Meshtastic.org, 18 Aug. 2024.

[7] N. K. Suryadevara and A. Dutta, "Meshtastic infrastructure-less networks for reliable data transmission to augment Internet of Things applications," in Wireless and Satellite Systems, Cham: Springer International Publishing, 2022, pp. 622–640.

[8] Sri Lanka Meteorology Department, Hydrological Annual Reports, 2014–2024. [Online]. Available: https://www.meteonet.gov.lk/. [Accessed: 22 Aug. 2025].

[9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Intern. Conf. Knowledge Discovery and Data Mining, San Francisco, CA, USA, 2016, pp. 785–794.

[10] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. New York: Springer, 2009.

[11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[12] S. Raschka, "Model selection and hyperparameter tuning," in Python Machine Learning, 3rd ed. Birmingham: Packt, 2021, ch. 6.

[13] M. Abadi et al., TensorFlow Lite: A framework for mobile and embedded machine learning, Google, 2019. [Online]. Available: https://www.tensorflow.org/lite. [Accessed: 22 Aug. 2025].

[14] A. P. Aung, "Flood prediction model using machine learning techniques," in Proc. IEEE Intern. Conf. on Big Data Analytics and Machine Learning (ICBDAML), Hanoi, Vietnam, 2019.

[15] K. R. Subrahmanyam and P. P. Kumar, "Real-time flood forecasting using IoT and machine learning," in Proc. IEEE Intern. Conf. Wireless and Mobile Networks (ICWMN), Mumbai, India, 2020.

[16] R. B. Gupta, "Hydrological modelling for flood risk assessment in tropical monsoon regions," Journal of Hydrology and Meteorology, vol. 45, no. 3, pp. 112–128, 2021.

[17] D. Ariyachandra, "ESP32-based environmental monitoring systems: A review of sensor integration techniques," IEEE Access, vol. 9, pp. 1542–1558, 2022.

[18] J. L. Smith, "Edge deployment of machine learning models on microcontroller platforms," in Proc. ACM Intern. Conf. Internet of Things (IoT), New York, USA, 2022.

[19] A. Kumar, S. Patel, and R. Mehta, "XGBoost vs. Logistic Regression for flood event classification: A comparative study," in Proc. IEEE Intern. Conf. Sustainable Development and Smart Cities (ICSSC), Kolkata, India, 2022.

[20] W. Das and A. Sen, "Web dashboard design for IoT-based disaster monitoring systems," in Proc. IEEE Intern. Conf. Advanced Technology in Industrial Application (ICATIA), Kuala Lumpur, Malaysia, 2023.